

Ranking Heuristics and Graph-Based Signals

We implemented three non-learned ranking heuristics as baselines: popularity-based ranking, item-item graph propagation, and Personalized PageRank.

Implementations

Popularity ranks items by how many users positively interacted with them. All users receive the same ranking modulo filtering of already-seen items.

Item-item graph propagation builds a co-occurrence matrix where edge weight between items i and j equals the number of users who positively interacted with both. For a target user, it scores candidates by summing co-occurrence weights from the user's history: $\text{score}(u, j) = \sum_{i \in \mathcal{I}_u} w(i, j)$.

Personalized PageRank runs a random walk with restart on the user-item bipartite graph. At each step the walker teleports back to the target user with probability α and follows a graph edge with probability $1 - \alpha$. With $\alpha = 0.15$, the walker follows edges 85% of the time, exploring multi-hop neighborhoods before teleporting back.

Results

Model	NDCG@10	MAP@10	Precision@10	Recall@10
Popularity	0.0762	0.0365	0.0642	0.0505
Item-Item Graph	0.0884	0.0429	0.0729	0.0631
Personalized PageRank	0.0782	0.0375	0.0657	0.0524

Item-item graph propagation leads on all ranking metrics. PPR performs marginally better than popularity but its degree normalization limits the advantage on this dense dataset.

Coverage

All three heuristics show very low catalog coverage, recommending only a small fraction of the ~3,700 item catalog.

Model	Unique Items Recommended	Coverage	Novelty
Popularity	~110	3.1%	8.37
Item-Item Graph	~120	3.3%	8.41
Personalized PageRank	~110	3.1%	8.37

Head vs. tail user breakdown

We split validation users by their training activity level (light: <50, moderate: 50-200, heavy: >200 interactions) to see where personalized methods pull ahead of popularity. The personalized heuristics (Graph, PPR) show

increasing advantage over the non-personalized popularity baseline as user activity increases, confirming that users with richer interaction histories benefit more from methods that leverage their history.

Let's break down these recommendation methods one at a time. I've reorganized the details so we can examine the main assumptions, the best use cases, and the drawbacks for each approach.

1. Popularity

Think of **Popularity** as the "everyone loves it, so you probably will too" method. This method is entirely non-personalized.

Popularity is helpful during a "cold start." When a new user joins and you have no data on them, a complex model will yield random results. Popularity offers a reliable baseline. Major platforms like Netflix, Spotify, and YouTube use a popularity fallback for this reason. It's also a good sanity check; if your complex algorithm can't outperform a simple popularity list, you may need to rethink your approach.

It overlooks niche preferences. Since everyone receives the same list, catalog coverage is poor, often around 3.1% (or just 110 out of 3,700 items). Additionally, in a live system, it creates a feedback loop: popular items get recommended, attracting more clicks, which makes them seem even more popular.

2. Item-Item Graph Propagation

This is the classic "customers who bought X also bought Y" method. It assumes items are relevant if they share many users with items you've interacted with.

This relies on a co-consumption principle. While it targets the user's history, it comes with a significant popularity bias. Highly popular items naturally connect to many other items in your catalog.

This method excels when data features a dense and meaningful co-occurrence structure. In a dataset like MovieLens, it often outperforms other simple rules because co-watching habits provide real signals.

The major flaw is that popular items often dominate the recommendations. Since they co-occur with many items, the algorithm tends to favor them, regardless of the user's original interests. Catalog coverage remains extremely low (around 3.3%), and it struggles to recommend "cold" items that haven't been interacted with.

3. Personalized PageRank (PPR)

Personalized PageRank seeks relevance by simulating a user randomly navigating the user-item graph, moving from one connection to the next.

PPR uses a graph proximity principle. With a dampening factor (like $\alpha = 0.15$), the simulated "walker" follows graph connections 85% of the time. This helps the algorithm find relevant items that are a few connections away rather than just direct connections. It normalizes the math to reduce the influence of ultra-popular nodes, addressing some biases found in basic item-item propagation.

PPR stands out in sparse, long-tail scenarios where direct connections are too rare to matter. By tracing multi-hop paths, it reveals hidden relationships between users and items.

It has difficulties if the graph is at either extreme. If it's disconnected, random walks become stuck in isolated clusters. If it's overly dense (like MovieLens), the walks spread too quickly, hitting mainly popular nodes, which

is why PPR performs just slightly better than raw popularity on that dataset. Lastly, it demands a lot of per-user computing power, making it noticeably slower in live production.

Limitations

The main issue with Popularity, Graph Propagation, and PPR is their rigidity. They apply fixed rules to fixed data. They assume they know what signals are useful.

This is why the industry is moving toward learned models (like BPR). Instead of making assumptions about the rules, a learned model optimizes directly for the ranking objective, allowing the system to discover the important patterns within the data.